

Video Transcripts

Module 16: Transit Data and APIs

Video 1: Introduction (04:30)

In this section, you have learned a lot about databases, all of the varieties of them. You've learned how to run them in containers. You've looked at multiple languages and how to run within those containers on those different platforms. You've looked at change data capture. So, now it's the time to apply this knowledge and we're going to ask you to do it on transit data from the city of Boston. This is a fun things to do. You're pulling data directly from the source. This is real data of real systems moving out in the world. And so, you will be making the decision of what database do I use? How do I handle data capture? How do I update the front end?

And in this case, we'll start you off with a stock one, but you can take all of this as always up to your own satisfaction and as far as you would like. The internet of things, IoT is the third generation of the web. The first one was about connecting documents. The second one was extending this connectivity to programs and data. And the third one is reaching out onto the physical world, the physical internet. And so, you can see here that a broad category of this third generation of the web is location-based applications. It turns out that there's a great deal of intelligence that can be derived and also great services that can be offered.

Now more specifically, we're going to be looking at bus tracking. And that is one of those things that if in areas like the Boston area on a cold winter night, especially if you have a storm, as you see here in the picture, you do not want to sit out in the cold. And now to be fair, also in places where it's very hot, like let's say the California desert, the area of death valley, the same thing would be true.

Now we're going to be taking a look at one specific route. This is Boston MBTA Route 1, MBTA stands for Massachusetts Bay Transportation Authority. And, this is a bus line that runs, it's a little bit more than that, but for practical purposes, we will define it as running between three university campus. And you can see there that it's between Harvard, MIT, and Boston University. And so once again, what we want to do is we want to be able to know where these buses are for

the convenience of users. We don't want people to freeze. And as mentioned as well, we want to capture some of that data. It would provide us to do analysis. It would allow those that are decision makers to see the performance, they would be able to see a historical record. And there are a number of advantages of doing that.

And so, the project that we're going to do is going to have the following architecture. You're going to have a web application that connects to a server that you will be writing. Now that server will be connecting to MBTA, and MBTA has provided an interface through which you can ask about the location of their buses. So, this is the portal to the developer portal to the services that are provided by MBTA. Now as mentioned, your part will be the server. We will provide you with some basic functionality on a web app that will connect to an API that you will have to write, an application programming interface that will be running on a server that you will be creating.

Now you will also be responsible for interrogating MBTA and requesting that information. So, more formally, your server will need to query MBTA, will need to store that data, will need to refresh that data periodically. It will need to build an API for clients, and it will need to serve that data on demand. Now once you've done the basics, you can feel free to expand onto the web application or expand on what you're doing on the server. There's a lot that can be done there, but we want you to get started first with the basics and then if you have time, you can do more.

Video 2: Using Time Loops to Show API Data (05:53)

Let's build a small simulation to explore the design decisions that you will need to make. As mentioned previously, the original tracking and the source of the data will be done by the MBTA and shared through their interfaces. When it comes to the front end, the client web app, we will provide you a basic version of that as well. So, your responsibility is that intermediate server, the one circle in red. So, to consider those choices and what you will be doing here, let's consider the following small simulation. We will be simulating the movement of four buses. As you can see there on the laptop, moving those buses across the screen.

We will have a time loop that will fire every 10 seconds. And every single time that fires, we will update the locations, and there are two files that we will share with you. The first one is `server.py`, the second one is `client`. Now we will also be able to use `cURL` at the command line to interrogate that API. Now before we move forward, it's worth mentioning that roughly speaking, we're working within that bounding box, which roughly captures the route that you will be

working with. Now in addition to that, the data format that is expected by clients is the following.

You can see there that we have the ID of the bus, latitude and longitude, which is the location and the direction of the bus as well. Now let's take a look at those files and we'll start off with `server.py`. As you can see here, there are a number of imports, `timer`, `flask`, `time` again and `Json`. These correspond to being able to do the loop, a basic web server, some timestamps and some data conversions. As you can see here, the very first block of code is the original locations of the buses. The second part here has to do with the increments that are added on each time loop.

We have a function here that updates the location and finally here what we have is a time loop that fires every 10 seconds and updates those locations. Then at the bottom here of the file, we have the web server. Most of it is your default basics for a server. And then, we have here a route that sends you to the HTML file. We will leave that for later. And then, we have the block that can provide you with the location. As you can see there, it simply takes the data structure, which is buses and the updates that may have been done to it and returns it as a `json.dump`.

So, let's go ahead and run this. But before we do, let's take a look at what the client does. And as you can see here, the client is very simple. All it does is it sets up an address, then it uses `request.urlopen` to read what comes back from that address, that is that location we are using this part here, that is the route that we're using within the Flask server to share our data about the buses, and then we simply write it to the screen. Now just as in the server, we have a time loop that will be repeating every 10 seconds and asking the server once again, what is the location of the buses?

So, let's take a look at this and see how that runs. So, I'm going to start off here by running the server that is `server.py`. And this is up and running now. And now, I'm going to go ahead and fire up the client. And so this is simply `client.py`. And as you can see there, I've started to write to the console the timestamps. So, now I'm going to go ahead and fire up the client. And as you can see there, the client is getting the data. And let's go ahead and let it run a little bit more just to see that information update.

And if you take a closer look at each of those latitudes and longitudes, you will notice that they have changed. So, before you go forward, take a look at this code. Make sure you understand what's happening. Make sure you understand what the client is doing as well. And I just noticed that we left out `cURL`. Let's go ahead and access it through `cURL` simply to illustrate that application programming interface, that API. Now as you can see here that continues to run. I'm

actually going to go ahead and stop the client and then I'm going to use cURL to head that address, it is going to be localhost. I'm running it in port 3000. And then, the path is location.

As you can see there, I get the location back just like I did when I was running the Python program. And I wanted to get it a little bit prettier here, but it doesn't really format any prettier. But if you look at the latitude and longitudes through each of those heads, depending on whether the data has been updated, I would have gotten a new set of coordinates. And after this, it should certainly be different, at least one of them. So, make sure you understand what's taking place here. This is a simulation, and so you can use it to explore. You're not hitting any external servers. It's all within your machine, although you already have the architectural boundaries that you will use later.

Video 3: Mapbox: A Web Client Library (03:55)

Next, let's take a look at the front end that the small web client that we have provided you. This is using a library, a great free library called 'mapbox. and it's supported by the chrome browser. Now before I go any further, I should say that if you do not change anything on this client, this will still work. But I'm going to provide you with the background and walk through the code in case you want to make any changes or you want to do any, any extensions to what we have provided you. So, let's take a look at that code now.

As you can see here, I have the index.html file. This is the file that will be served if you hit the root of the application by flask. The top of the file brings in the mapping libraries from mapbox. It establishes some style and then the rest of this file defines the application. You can see here that you will need a token. In this point, I have simply written in there your token, but you can get your actual token from this address. I had mine before, but I updated it with your token simply to illustrate that you will need your own.

This is a great one to have, as mentioned it's free, and you can use these maps on any application you like. Now most of the application involves querying the server, that is asking for updated information from the server and then updating the location of the buses on the screen. Now you can see here that there are a couple of global variables. The first one is map, the second one is markers which will hold all of the buses locations in the visual marker on the screen. We then create a map and this is aided by that library and then we initialize the application by adding markers.

The very first thing that happens is that we get the bus locations. This calls the location on the server/location and brings back that json data. Then that is parsed and we return an object to this addMarkers function. The very first thing that we do after that is that then we loop through all of the buses that have been sent. If we find the marker for it, we simply update its location as you can see here, move marker. Otherwise we add a new marker to the screen. Now below this, there are all the functions to support that and you can read through those on your own.

Now let's fire up the server, bring up the web client, and take a look at that and see what it looks like. So, let's fire up the server. As you can see there, that's up and running. Let's open up the browser and the address is localhost:3000 and that's 3000. And as you can see there, this is monstrous. Let's go ahead and make the size on this a little bit more reasonable. Go ahead and move in here. And if we took a look at it for a little while, you see the move. You can see there that they just move.

The other thing to note as well is that they're not following precisely route one, because this is simply doing a linear mapping from one end of that bounding box to the other. But it's a pretty good one here to simply understand what's taking place. You're still working with the simulated server. If you wait for them to get all the way to the end and come back, you'll see them change color as well. So, this is a good one to get understanding on and then, once you do, you can go forward.

Video 4: Mapbox Overview (05:53)

Maps are a great example of bringing in resources onto our page and mixing them up with our UI. We can bring in the libraries, we can bring in the styles, and we can layout great resources in this way. In this case, we're going to bring in maps, and so we're going to start off by bringing in the library, and that is Mapbox.com's library as you can see there, and we're using this one because it allows you over 15,000 or 50,000 actually views per month for free. This is the library. Now, I'm going to go ahead and bring in the styles.

This brings in the styles for the mapping library, and we can view either one of these by navigating directly in the browser. So, in this case, we're looking at the mapping library, and you can see there how big it is, even though it's minimized. And I could do the exact same thing for the styles by accessing them directly. So, by doing those includes in the HTML document, we're loading all of those resources, and they will now be available for us to use within our page.

Next, I'm going to add some styles. I'm going to set some styles for the body because I will be laying out a map. I want my margins and my padding to be zero, to make sure I have no spacing around the map. Then, the styles for the map element are going to be the following. I'm going to set my position to be absolute, my top at zero, bottom at zero, and my width to be a 100%. Now within the body of the document, I'm going to create the element that will hold the map and I'm going to use a div for this, and the ID of the div is going to be map. When we write our code in our script block, we will reference that div. We're now going to use a function within the mapping library to create a new map. And so I'm going to use 'new' Mapboxgl.Map(), and this creates, this function creates a new map, and that function takes an object that has a certain number of properties. So, the first one is 'container'. And here is where we reference '.div' ID of map.

And so, I'm going to enter that here. Next, I'm going to set the style, and this is defined by the library and the styles makers which in this case is mapbox. So, I'm going to go ahead and paste that in here. Next, we're going to set the starting location, which is going to be the longitude and latitude. And this is one thing to watch, depending on which library you're using, you could get latitude first or longitude first. And then, the last thing that we're going to set is the initial zoom on the page. So, in this case, we're going to set the initial zoom at 10. And we will change these in a moment once we load the map. But there's one more thing that we need. The additional thing that we need is that we need an access token.

And here, I will refer you to account.mapbox.com for you to get your own access token. I will not share mine because since these videos are viewed by many, they would stop working soon after the 50,000 limit per month is reached. So, I'm going to go ahead and paste my token here and now let's go ahead and load up the map. So, I'm going to go ahead and reload this page and as you can see there we are viewing the city of Boston. If I was to zoom in, let's say we use 15 here, I'm going to go ahead and reload the page, then you could see that we are zoomed in to Cambridge and we are at MIT's campus. I could change the styles as well. Let's say we go with a dark style instead of the default one that I'm using here, and let me go ahead and remove all of it and paste the new style. And now, I'm going to go ahead and reload the page. And you'll see there that now the styles are very different.

So, this is an example of basic mapping. If you use the Google Maps API, you would do something similar, as I mentioned as well, Bing would do it as well, and many others. You can see here some

of the elements are bringing in the library, bringing in the styles, setting some of the styles around the element that you're going to set. We talked about margining and padding, and other things, and then creating the function or using the function that is going to create the map, giving it the element within the page, in this case, the div id map that where you're going to position your map. And then some of the initial settings as well, which are the level of zoom, the location and latitude and longitude, the style that you're going to use, and in this case the container. So, give it a try. This is something that is very useful in many different contexts, and as you can see, you can control this programmatically.

Video 5: Mapbox: Adding a Marker (02:38)

As in our previous example, we have loaded the JavaScript libraries for mapbox. We have done the styles as well and then we have set some styling for the page. The body doesn't have any margin or padding. And then we have set the following styles for the element that is going to hold the map. And you can see the element here, which is div id of map. We then set our access token. And once again here you would have to get that access token yourself. And you can do so by navigating to mapbox and getting your own personal access token.

We then create a new map and you can see here the parameters that we're passing in and the object which are the container map, the tag that is going to be held in, in this case, the div of id map, then the styles, then the center of the map, which is as you can see here. The last thing is the zoom, the initial zoom. And so that is what we did before, you can see the map here on the right-hand side and the browser. And now what we're going to do is that we're going to set up or we're going to add to this map a marker and we're going to use it, we're going to do it using a function from the mapbox library. And so we will enter here `mapboxgl.Marker`, and that's the function.

Then we will, in addition to that, set the longitude and latitude. So, let's set 'LngLat' and then the location that we're going to set is going to be the following. Let me go ahead and enter that in. After that we're going to add it to the map 'addTo(map)'. And now, let's go ahead and reload the page on the right-hand side and we should get a marker around here. Let's go ahead and see. And yes, you can see there that we did in fact get a marker. And if we zoom in a bit on the location, you can see there that that is the MIT campus. So, to recap, we created a map like we had done before, and in addition to that we added a marker which indicates a location, in this case the MIT campus.

Video 6: The MBTA API (03:38)

When it comes to cities, the MBTA API, is a pretty friendly one. You can navigate to `api-v3.mbtta.com`, and register as a developer, and get a key. But for our purposes, experimentation, we can really go without any keys as you can see here. You can use the following URL to go directly without registering, and use the following path to pull back the data of route 1. You can see there, that the very first part of the URL refers to vehicles, the MBTA also has trains, ferries, and subways. And that, in this case, we don't just want all vehicles; we want to filter to route 1 and we want to include the trip.

So, let's take a look at this address in the browser. Let's paste it on the address bar and as you can see there, we have a list of buses in its data. This is live at this specific second, the information that is coming back. You should also note that in case you are on time zones where you might have the API at times that no buses are running, in which case you will not get any data because there are no buses on the road. So, take that into account.

Now, if we were to search here for latitude and longitude, you can see here that we would start to get those for a number of buses, if we scroll up and down. Now you could also see that there is some information, for example, about seating. You can see there are a few seats available, many seats available, and so on, for the data. So, as you can see there, we can go directly to that address and pull back the information for that route.

And so, the next thing that we're going to do is that we're going to write a very simple client that is simply going to hit that endpoint, and bring back that data and write it to the console. As you can see here, this is the simple program. It has the address or the URL that we just used. It then pulls it back using `request.urlopen`. Since this is JSON data, it parses it using `json.loads`, and then we're simply going to write that to the console. So, here I have an empty file that I've just created, it's called `mbta_client.py`. Into it, I'm going to paste the URL information and I'm going to uncomment this part here, and I see what happened.

My syntax got broken up there from doing the copy and paste from PowerPoint. And now, I have it within the file, I'm going to go ahead and save it. I'll go ahead and open up the terminal. As you can see there, I'm going to go ahead and move down a little bit so we can see all of our code. I'm then going to go ahead and enter `python3`, the name of my file, and let's go ahead and run it. And as you can see there, we got the exact same data that we did. Well, it's not the exact same

one because time has gone by and the buses most likely have moved.

But we had the same behavior that we did when we accessed through the browser using the address. But now, we're doing that programmatically, and you can see there, that we're parsing that information there as well. So, you can use that same approach on your server when you are polling and requesting information from MBTA.

Video 7: Project 1: Build a Transit Data Application (02:19)

It is now your turn. We have seen how you can request information from MBTA. We've also seen a basic front-end client that will render your locations as long as you give it to them in the right format. It is now your turn to build that middle tier, that server that is going to provide tracking for buses. Choose your database, choose your logic, figure out how you're going to do polling, and build your application. Now, this should be a lot of fun. You're bringing together a lot of the different concepts that we have seen in the course. Now once you have that under control and you're happy with it, there's a bunch of other things that you could do. These are completely optional, but I figure I would mention a few just in case you wanted to do that stretch.

When it comes to the UI, there's a bunch of improvements that you could do. For example, you could warn users when there is congestion, when buses are running slow. You could also do some improvements when it comes to the direction. You could put an arrow, say, on each of your buses as they're moving. You could warn people on the occupancy, how full the bus is. You could do that visually as well and you can see there are some other suggestions. Now you could look at it as well from the analysis perspective. When do I expect this bus to arrive, giving a certain location. You could do something similar when it comes to the times and days of the week and for example, when would be the best time to take a bus, if you could take it at mid morning.

Now you can see there a last one at the very bottom of the page, that if you captured the data of buses for 24 hours, then you could create an interesting visualization that moved through the hours of the day at an accelerated pace, to be able to get a visual sense of how traffic moves in the city. Now some of these could be quite valuable when it comes to understanding traffic patterns. So, there's a number of suggestions. These are optional once again, but these are some that might interest you and might be fun.